

BirdNest Website — Status Report

Date: 22 June 2026

Prepared for: Management & Development Team

Project: BirdNest Holiday Rental Platform

WHAT IS THIS WEBSITE?

BirdNest is a holiday rental booking platform. Clients visit the site, browse properties, and book stays online with payment taken at checkout. The site supports **English and Arabic**, runs on Vercel (cloud hosting), stores all data in a PostgreSQL database (Supabase), and processes payments through **Stripe**.

STRENGTHS — WHAT IS BUILT AND WORKING

The following are complete, tested, and solid foundations the team can be confident in.

- 1 The visual website looks complete.** All customer-facing pages are designed and polished: homepage, listings browse, property detail, 3-click checkout, account dashboard (sign-in, register, booking history), compare, wishlist, contact, blog, how-it-works, and terms. Branding — colours, fonts, and logos — is consistent throughout.
- 2 User accounts are fully built.** Customers can register with email and password, or sign in with Google. Passwords are stored securely using industry-standard hashing. Session management works correctly, and the account dashboard shows a logged-in user's bookings.

- 3 **The live database is set up and seeded.** A production database exists in the EU (Supabase). All 8 data tables are created (Users, Listings, Bookings, Reviews, etc.), 12 real property listings and 36 reviews are loaded, and row-level security is enabled — meaning data is locked down from unauthorised access.
- 4 **The full backend API is written.** All server-side code is complete: fetching listings, search, creating bookings, processing Stripe payments, handling Stripe webhooks (payment confirmations and refunds), and admin tools. The engine is built — it just needs to be connected to the front end.
- 5 **Security foundations are strong.** The site sends all major security headers (prevents clickjacking, content injection, etc.), enforces HTTPS, has a Content Security Policy allowlisting only trusted services, and rate-limits the registration page against brute-force attacks. The codebase builds with zero TypeScript errors and zero lint warnings.
- 6 **The build is clean and deployable.** The code compiles and passes all checks. From a purely technical standpoint it can be pushed live at any time.

WEAKNESSES — WHAT IS INCOMPLETE

CRITICAL — The Site Cannot Take Real Bookings Yet

#	Issue
1	Pages show fake demo data, not the real database. When a customer visits the site today, they see hardcoded mock properties — not the 12 real listings stored in the database. All browse, detail, booking, compare, and wishlist pages read from a demo data file in the code. The API that reads the real database is already written; it just has not been connected to the front-end pages yet.
2	The checkout does not process a real payment. Clicking "Confirm & Pay" runs a fake timer. No card is charged, no booking is saved in the database, and no confirmation email is sent. The Stripe payment code and booking creation API both exist — they just haven't been wired into the checkout page yet.
3	

#	Issue
---	-------

Secret keys have not been set in the hosting environment. The live website on Vercel does not have the credentials it needs: database connection, authentication secret, and Stripe keys. Without these, the site will fail at runtime even once the code is connected. These must be entered manually in the Vercel dashboard for security reasons.

MODERATE — Important, But Not Hard Blockers for Launch

#	Issue
---	-------

4 Database migration records are out of sync. The database was set up by running the full schema directly rather than through the step-by-step migration files in the code. If a developer ever creates a fresh database using the standard process, one column (guest nationality on the Bookings table) would be missing. A small fix is needed to keep the code and database in agreement.

5 The brute-force rate limiter doesn't work across multiple servers. Because the site runs on Vercel's serverless infrastructure (many parallel instances), the current rate limiter — which stores its counter in a single server's memory — can be bypassed. It should be replaced with a shared cloud-based solution before handling real users.

ADDITIONAL CONCERNS NOT PREVIOUSLY COVERED

#	Concern
---	---------

6 No independent security audit has been done. The code has good security practices, but no external penetration test has been performed. Before taking real payments from real clients, this is strongly recommended — payment card data and personal information will be at stake.

7 No real domain name has been connected. The site currently lives at a Vercel auto-generated URL. The actual domain name (e.g. birdnest.com or similar) has not been pointed to the site. DNS configuration is still outstanding.

8 Email confirmations have not been tested end-to-end. A Resend email integration exists in the code, but whether booking confirmation and

#	Concern
	password-reset emails actually send and deliver correctly has not been verified with a real test.
9	The Arabic version has not been reviewed. The site supports English and Arabic. However, there is no record of the Arabic translation content being checked for accuracy or completeness by a native speaker.
10	No admin panel exists for the property owner. There is no management area to add properties, update availability, view bookings, or adjust pricing. Currently all of that requires a developer to access the database directly — which is not sustainable.

WHAT MUST BE DONE BEFORE GOING LIVE

Minimum required tasks, in recommended order. Completing all six makes the site ready to accept real client bookings.

#	Task	Who	Effort
1	Connect listing, detail, booking, compare, and wishlist pages to the live database — replace demo data with real API calls	Developer	1–2 days
2	Wire real Stripe payment and booking creation into the checkout page	Developer	1 day
3	Set the secret environment variables in Vercel dashboard (database URL, auth secret, Stripe keys, email key)	Owner	1 hour
4	Fix the database migration drift (add the missing nationality migration)	Developer	2 hours
5	End-to-end test: make a real test booking, confirm Stripe charge, confirm booking appears in account dashboard	Developer + Owner	2–4 hours
6	Connect the real domain name (update DNS to point to Vercel deployment)	Owner / Developer	1–2 hours

Recommended Before Opening to the Public

Not hard blockers, but important for a professional and secure launch.

#	Task
A	Replace in-memory rate limiter with a cloud-based Redis solution (Upstash)
B	Commission an independent security review / light penetration test
C	Test all email confirmations (booking confirmed, password reset) end-to-end
D	Review Arabic translation content with a native speaker
E	Build a basic admin panel so the owner can manage listings and bookings without developer help

Summary



The BirdNest website is well-built and approximately **60–70% complete**. The visual design, user accounts, live database, full backend API, and security foundations are all solid. What remains is primarily **connecting the pieces** — the front-end pages need to read from the real database, the checkout needs to process real payments, and the Vercel environment keys need to be entered. None of these are large or complex tasks; they are wiring jobs.

With focused developer time, the site could be ready to take its **first real client booking within approximately one week**.